

续表

(3) 语句级审计

- ① 对表定义的更改语句 ALTER 设置审计。

```
AUDIT ALTER TABLE BY ACCESS;
```

- ② 查看数据库所有语句级审计设置，验证审计设置是否生效。

```
SELECT * FROM SYS_STMT_AUDIT_OPTS;
```

- ③ 以普通用户登录，执行 SQL 语句，验证审计设置是否生效。

```
ALTER TABLE Customer ADD COLUMN tt INT;
```

- ④ 查看所有审计信息。

```
SELECT * FROM SYS_AUDIT_TRAIL;
```

实验总结：

① 在 KingbaseES 系统中，只有审计管理员（SAO）才能进行对象级、语句级、系统权限级等审计权限设置。通过系统视图 SYS_STMT_AUDIT_OPTS 可以查看所有数据库中语句级审计设置，通过系统视图 SYS_AUDIT_OBJECT 可以查看数据库对象级审计设置。

② 通过系统视图 SYS_AUDIT_STATEMENT 可以查询所有关于 GRANT、REVOKE、AUDIT、NOAUDIT 以及 ALTER SYSTEM 语句的审计结果；通过系统视图 SYS_AUDIT_OBJECT 可以查询数据库中所有对象的审计结果；通过系统视图 SYS_AUDIT_TRAIL 可以查看所有审计记录。

- ③ 审计语句不是标准 SQL 语句，所以不同的系统语句格式和语法不尽相同。

3. 思考题

- ① 请设计一个示例，分析数据库审计对数据库性能的影响情况。
② 请分析审计与数据库日志的异同。

第5章实验 完整性控制

完整性控制实验包含 4 个实验项目（见表 6），其中 3 个必修实验项目，1 个选修实验项目。本章实验的各个实验项目均为验证性实验。

表 6 “完整性控制”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应教材章节
实验 5.1	实体完整性实验	必修	适中	2	第 5 章 5.2 节
实验 5.2	参照完整性实验	必修	适中	2	第 5 章 5.3 节
实验 5.3	用户自定义完整性实验	选修	适中	2	第 5 章 5.4 节
实验 5.4	触发器实验	必修	较难	2	第 5 章 5.7 节

实验 5.1 实体完整性实验

1. 实验内容和要求

掌握实体完整性的定义和维护方法。定义实体完整性，删除实体完整性；能够写出两种方式定义实体完整性的 SQL 语句：创建表时定义实体完整性、创建表后定义实体完整性；设计 SQL 语句以验证完整性约束是否起作用。

实验重点：创建表时定义实体完整性。

实验难点：当有多个候选码时，如何定义实体完整性。

2. 实验报告示例

实验报告			
题目：实体完整性实验	姓名	日期	
(1) 创建表时定义实体完整性（列级实体完整性） 定义供应商表的实体完整性。 <pre>CREATE TABLE Supplier(Suppkey INTEGER CONSTRAINT PK_supplier PRIMARY KEY, Name CHAR(25), Address VARCHAR(40), Nationkey INTEGER, Phone CHAR(15), Acctbal REAL, Comment VARCHAR(101));</pre>			
(2) 创建表时定义实体完整性（表级实体完整性） 定义供应商表的实体完整性。 <pre>CREATE TABLE Supplier (Suppkey INTEGER, Name CHAR(25), Address VARCHAR(40),</pre>			

续表

```
Nationkey INTEGER,  
Phone CHAR(15),  
Acctbal REAL,  
Comment VARCHAR(101),  
CONSTRAINT PK_supplier PRIMARY KEY(Suppkey) );
```

(3) 创建表后定义实体完整性

定义供应商表。

```
CREATE TABLE Supplier (  
    Suppkey INTEGER,  
    Name CHAR(25),  
    Address VARCHAR(40),  
    Nationkey INTEGER,  
    Phone CHAR(15),  
    Acctbal REAL,  
    Comment VARCHAR(101) );
```

```
ALTER TABLE Supplier      /*再修改供应商表，增加实体完整性*/  
ADD CONSTRAINT PK_Supplier PRIMARY KEY(Suppkey);
```

(4) 定义实体完整性（主码由多个属性组成）

定义供应关系表的实体完整性。

```
CREATE TABLE PartSupp (  
    Partkey INTEGER,  
    Suppkey INTEGER,  
    Availqty INTEGER,  
    Supplycost REAL,  
    Comment VARCHAR(199),  
PRIMARY KEY(Partkey,Suppkey) ;  
/*主码由多个属性组成，实体完整性必须定义在表级*/
```

(5) 有多个候选码时定义实体完整性

定义国家表的实体完整性，其中 Nationkey 和 Name 都是候选码。选择 Nationkey 作为主码，在 Name 上定义唯一性约束。

```
CREATE TABLE Nation (  
    Nationkey INTEGER CONSTRAINT PK_nation PRIMARY KEY,  
    Name CHAR(25) UNIQUE,
```



```
Regionkey INTEGER,  
Comment VARCHAR(152) );
```

(6) 删除实体完整性

删除国家实体的主码。

```
ALTER TABLE Nation DROP CONSTRAINT PK_nation;
```

(7) 增加两条相同记录, 验证实体完整性是否起作用

/*插入两条主码相同的记录就会违反实体完整性约束*/

```
INSERT INTO Supplier (Suppkey,Name,Address,Nationkey,Phone,Acctbal,Comment )  
VALUES (11,'test1','test1',101,'12345678',0.0,'test1' );
```

```
INSERT INTO Supplier (Suppkey,Name,Address,Nationkey,Phone,Acctbal,Comment )  
VALUES (11,'test2','test2',102,'23456789',0.0,'test2' );
```

实验总结:

① 完整性约束既可以在创建表时定义, 也可以在创建表之后定义, 利用 ALTER TABLE 语句增加、修改或者删除实体完整性。

② 当表中已有数据违反完整性约束时, 完整性约束就不能成功建立, 需要“清洗”表中已有数据, 使之符合规定的完整性约束, 然后才能成功建立相应的完整性约束。

3. 思考题

① 所有列级完整性约束都可以改写为表级完整性约束, 而表级完整性约束不一定能改写成列级完整性约束。请举例说明。

② 什么情况下会违反实体完整性约束? 违反实体完整性约束时, DBMS 将做何种违约处理? 请用实验验证。

实验 5.2 参照完整性实验

1. 实验内容和要求

掌握参照完整性的定义和维护方法。定义参照完整性, 定义参照完整性的违约处理, 删除参照完整性; 用两种方式写出定义参照完整性的 SQL 语句: 创建表时定义参照完整性、创建表后定义参照完整性。

实验重点: 创建表时定义参照完整性。

实验难点: 定义参照完整性的违约处理。

2. 实验报告示例

实验报告

题目：参照完整性实验

姓名

日期

(1) 创建表时定义参照完整性

先定义地区表的实体完整性，再定义国家表上的参照完整性。

```
CREATE TABLE Region(  
    Regionkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Comment VARCHAR(152));  
  
CREATE TABLE Nation(  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Regionkey INTEGER REFERENCES Region(Rregionkey),    /*列级参照完整性*/  
    Comment VARCHAR(152));
```

或者：

```
CREATE TABLE Nation(  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Regionkey INTEGER,  
    Comment VARCHAR(152),  
    CONSTRAINT FK_Nation_regionkey FOREIGN KEY (Regionkey) REFERENCES  
        Region(Rregionkey);    /*表级参照完整性*/
```

(2) 创建表后定义参照完整性

定义国家表的参照完整性。

```
CREATE TABLE Nation (  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Regionkey INTEGER,  
    Comment VARCHAR(152));  
  
ALTER TABLE Nation  
ADD CONSTRAINT FK_Nation_regionkey  
FOREIGN KEY(Rregionkey) REFERENCES Region(Rregionkey );
```

(3) 定义参照完整性（外码由多个属性组成）

定义订单明细表的参照完整性。

续表

```
CREATE TABLE PartSupp (  
    Partkey INTEGER,  
    Suppkey INTEGER,  
    Availqty INTEGER,  
    Supplycost REAL,  
    Comment VARCHAR(199),  
    PRIMARY KEY(Partkey,Suppkey)) ;  
  
CREATE TABLE Lineitem (  
    Orderkey INTEGER REFERENCES Orders(Orderkey),  
    Partkey INTEGER REFERENCES Part(Partkey),  
    Suppkey INTEGER REFERENCES Supplier(Suppkey),  
    Linenumber INTEGER,  
    Quantity REAL,  
    Extendedprice REAL,  
    Discount REAL,  
    Tax REAL,  
    Returnflag CHAR(1),  
    Linestatus CHAR(1),  
    Shipdate DATE,  
    Commitdate DATE,  
    Receiptdate DATE,  
    Shipinstruct CHAR(25),  
    Shipmode CHAR(10),  
    Comment VARCHAR(44),  
    PRIMARY KEY(Orderkey,Linenumber),  
    FOREIGN KEY (Partkey,Suppkey) REFERENCES PartSupp(Partkey,Suppkey));
```

(4) 定义参照完整性的违约处理

定义国家表的参照完整性, 当删除或修改被参照表记录时, 设置参照表中相应记录的值
为空值。

```
CREATE TABLE Nation (  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Regionkey INTEGER,  
    Comment VARCHAR(152),  
    CONSTRAINT FK_Nation_regionkey FOREIGN KEY (Regionkey) REFERENCES  
    Region(Regionkey) ON DELETE SET NULL ON UPDATE SET NULL );
```


续表

(5) 删除参照完整性

删除国家表的外码。

```
ALTER TABLE Nation DROP CONSTRAINT FK_Nation_regionkey;
```

(6) 插入一条国家记录, 验证参照完整性是否起作用

```
/*插入一条国家记录, 如果'1001'号地区记录不存在, 违反参照完整性约束。*/
```

```
INSERT INTO Nation(Nationkey,Name,Regionkey,Comment )
VALUES (10,'nation1',1001,'comment1');
```

实验总结:

参照完整性通常涉及两个表, 要区分清楚引用表和被引用表、引用属性和被引用属性。对建立参照完整性约束的两个表进行数据更新时, 要清楚什么情况下可能导致参照完整性约束违约。

3. 思考题

对于自引用表, 例如, 课程表(课程号, 课程名, 先修课程号, 学分)中的先修课程号引用该表的课程号, 请完成如下任务:

- ① 写出课程表上的实体完整性和参照完整性。
- ② 在考虑参照完整性约束的情况下, 列举出几种录入课程数据的方法。

实验 5.3 用户自定义完整性实验**1. 实验内容和要求**

掌握用户自定义完整性的定义和维护方法。针对具体应用语义, 选择 NULL/NOT NULL、DEFAULT、UNIQUE、CHECK 等定义属性上的约束条件。

实验重点: NULL/NOT NULL、DEFAULT 约束。

实验难点: CHECK 约束。

2. 实验报告示例

实验报告			
题目: 用户自定义完整性实验	姓名	日期	
(1) 定义属性 NULL/NOT NULL 约束 定义地区表各属性的 NULL/NOT NULL 属性。 <pre>CREATE TABLE Region (Regionkey INTEGER PRIMARY KEY, Name CHAR(25) NOT NULL,</pre>			

续表

```
Comment VARCHAR(152) );
```

(2) 定义属性 DEFAULT 约束

定义国家表的 Regionkey 的缺省属性值为 0，表示其他地区。

```
CREATE TABLE Nation (  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25),  
    Regionkey INTEGER DEFAULT 0,  
    Comment VARCHAR(152),  
    CONSTRAINT FK_Nation_regionkey FOREIGN KEY (Regionkey) REFERENCES  
    Region(Regionkey));
```

(3) 定义属性 UNIQUE 约束

定义国家表的名称属性必须满足唯一的完整性约束。

```
CREATE TABLE Nation (  
    Nationkey INTEGER PRIMARY KEY,  
    Name CHAR(25) UNIQUE,  
    Regionkey INTEGER,  
    Comment VARCHAR(152));
```

(4) 使用 CHECK

使用 CHECK 定义订单明细表中某些属性应该满足的约束。

```
CREATE TABLE Lineitem (  
    Orderkey INTEGER REFERENCES Orders(Orderkey),  
    Partkey INTEGER REFERENCES Part(Partkey),  
    Suppkey INTEGER REFERENCES Supplier(Suppkey),  
    Linenumber INTEGER,  
    Quantity REAL,  
    Extendedprice REAL,  
    Discount REAL,  
    Tax REAL,  
    Returnflag CHAR(1),  
    Linestatus CHAR(1),  
    Shipdate DATE,  
    Commitdate DATE,  
    Receiptdate DATE,  
    Shipinstruct CHAR(25),  
    Shipmode CHAR(10),  
    Comment VARCHAR(44),
```


续表

```
PRIMARY KEY(Orderkey,Linenumber),
FOREIGN KEY (Partkey,Suppkey) REFERENCES PartSupp(Partkey,Suppkey),
CHECK(Shipdate < Receiptdate),           /*装运日期<签收日期*/
CHECK(Returnflag IN ('A','R','N')));      /*退货标记为 A 或 R 或 N*/
```

(5) 修改 Lineitem 的一条记录以验证是否违反 CHECK 约束

```
UPDATE Sales.Lineitem SET Shipdate = '2015-01-05', Receiptdate = '2015-01-01'
WHERE Orderkey = 5005 AND Linenumber = 1;
```

实验总结:

① 用户自定义完整性约束要根据应用需求确定, NULL/NOT NULL 是常用的用户自定义完整性约束,但对于主码,系统自动设置为 NOT NULL。

② CHECK 约束可以建立同一个表的一个或者多个属性之间的完整性约束,对保证数据的一致性具有重要作用。通常应该在数据库模式上定义完整性约束,以充分利用数据库的完整性约束检查机制和能力,而不是在应用程序中定义数据的完整性约束,否则将极大地增加应用程序设计和实现的复杂性和工作量。

3. 思考题

- ① 请分析哪些完整性约束只针对单个属性,哪些完整性约束可以针对多个属性;哪些只针对一个表,哪些针对多个表。
- ② 对表中某一列数据类型进行修改时,要修改的列是否必须为空列?

实验 5.4 触发器实验

1. 实验内容和要求

掌握数据库触发器的设计和使用方法。定义 **BEFORE** 触发器和 **AFTER** 触发器;能够理解不同类型触发器的作用和执行原理,验证触发器的有效性。

实验重点: 触发器的定义。

实验难点: 利用触发器实现较为复杂的用户自定义完整性。

2. 实验报告示例

实验报告			
题目: 触发器实验	姓名	日期	
(1) AFTER 触发器			
在 Lineitem 表上定义一个 UPDATE 触发器,当修改订单明细(即修改订单明细价格 Extendedprice,折扣 Discount,税率 Tax)时,自动修改订单 Orders 的 TotalPrice,以保持数据一致性, $Totalprice = Totalprice + Extendedprice * (1 - Discount) * (1 + Tax)$ 。			

续表

```

CREATE OR REPLACE TRIGGER TRI_Lineitem_Price_UPDATE
AFTER UPDATE OF Extendedprice,Discount,Tax ON Lineitem
FOR EACH ROW
AS
DECLARE
    L_valuediff REAL;
BEGIN
    /*订单明细修改后, 计算订单含税折扣价总价的修正值*/
    L_valuediff=NEW.Extendedprice*(1-NEW.Discount)*(1+NEW.Tax) -
    OLD.Extendedprice*(1-OLD.Discount)*(1+OLD.Tax);
    /*更新订单的含税折扣价总价*/
    UPDATE Orders SET Totalprice = Totalprice + L_valuediff
    WHERE Orderkey = NEW.Orderkey;
END;

```

(2) INSERT 触发器

在 Lineitem 表上定义一个 INSERT 触发器, 当增加一项订单明细时, 自动修改订单 Orders 的 TotalPrice, 以保持数据一致性。

```

CREATE OR REPLACE TRIGGER TRI_Lineitem_Price_INSERT
AFTER INSERT ON Lineitem
FOR EACH ROW
AS
DECLARE
    L_valuediff REAL;
BEGIN
    L_valuediff = NEW.Extendedprice*(1 - NEW.Discount)*(1 + NEW.Tax);
    /*增加订单明细项后, 计算订单含税折扣价总价的修正值*/
    UPDATE Orders SET Totalprice = Totalprice + L_valuediff
    /*更新订单的含税折扣价总价*/
    WHERE Orderkey = NEW.Orderkey;
END;

```

(3) DELETE 触发器

在 Lineitem 表上定义一个 DELETE 触发器, 当删除一项订单明细时, 自动修改订单 Orders 的 TotalPrice, 以保持数据一致性。

```

CREATE OR REPLACE TRIGGER TRI_Lineitem_Price_DELETE
AFTER DELETE ON Lineitem
FOR EACH ROW

```

续表

```
AS
DECLARE
L_valuediff REAL;
BEGIN
    L_valuediff = - OLD.Extendedprice*(1 - OLD.Discount)*(1 + OLD.Tax);
    /*删除订单明细项后, 计算订单含税折扣价总价的修正值*/
    UPDATE Orders SET Totalprice = Totalprice + L_valuediff
    /*更新订单的含税折扣价总价*/
    WHERE Orderkey = NEW.Orderkey;
END;
```

(4) 验证触发器 TRI_Lineitem_Price_UPDATE

```
/*查看 1854 号订单的含税折扣总价 Totalprice*/
SELECT Totalprice
FROM Orders
WHERE Orderkey = 1854;
/*激活触发器: 修改 1854 号订单第一个明细项的税率, 该税率增加 0.5%*/
UPDATE Lineitem SET Tax = Tax + 0.005
WHERE Orderkey = 1854 AND Linenumber = 1;

/*再次查看 1854 号订单的含税折扣总价 Totalprice 是否有变化, 如有变化则是触发器起作用了,
否则触发器没有起作用*/
SELECT Totalprice
FROM Orders
WHERE Orderkey = 1854;
```

(5) BEFORE 触发器

① 在 Lineitem 表上定义一个 BEFORE UPDATE 触发器, 当修改订单明细中的数量 (Quantity) 时, 先检查零件供应表 PartSupp 中的可用数量 Availqty 是否足够。

```
CREATE OR REPLACE TRIGGER TRI_Lineitem_Quantity_UPDATE
BEFORE UPDATE OF Quantity ON Lineitem
FOR EACH ROW
AS
DECLARE
    L_valuediff INTEGER;
    L_availqty INTEGER;
BEGIN
    /*计算订单明细项修改时, 订购数量的变化值*/
    L_valuediff = NEW.Quantity - OLD.Quantity;
```


续表

```

/*查询当前订单明细项对应零件供应记录中的可用数量*/
SELECT Availqty INTO L_availqty
FROM PartSupp
WHERE Partkey = NEW.Partkey AND Suppkey = NEW.Suppkey;

IF (L_availqty - L_valuediff >= 0) THEN
    BEGIN
        /*如果可用数量可以满足订单订购数量，则提示 ENOUGH*/
        RAISE NOTICE 'Available Quantity is ENOUGH';
        /*修改当前订单明细项对应零件供应记录中的可用数量*/
        UPDATE PartSupp
        SET Availqty = Availqty - L_valuediff
        WHERE Partkey = NEW.Partkey AND Suppkey = NEW.Suppkey;
    END;
ELSE
    /*如果可用数量不能满足订单订购数量，则更新过程异常中断*/
    RAISE EXCEPTION 'Available Quantity is NOT ENOUGH';
END IF;
END;

```

② 在 Lineitem 表上定义一个 BEFORE INSERT 触发器，当插入订单明细时，先检查零件供应表 PartSupp 中的可用数量 Availqty 是否足够。

```

CREATE OR REPLACE TRIGGER TRI_Lineitem_Quantity_UPDATE
BEFORE INSERT ON Lineitem
FOR EACH ROW
AS
DECLARE
    L_valuediff INTEGER;
    L_availqty INTEGER;
BEGIN
    L_valuediff = NEW.Quantity;          /*获得插入订单明细项的订购数量*/
    /*查询当前订单明细项对应零件供应记录中的可用数量*/
    SELECT Availqty INTO L_availqty
    FROM PartSupp
    WHERE Partkey = NEW.Partkey AND Suppkey = NEW.Suppkey;

    IF (L_availqty - L_valuediff >= 0) THEN
        BEGIN
            /*如果可用数量可以满足订单订购数量，则提示 ENOUGH*/

```


续表

```

        RAISE NOTICE 'Available Quantity is ENOUGH';
        /*修改当前订单明细项对应零件供应记录中的可用数量*/
        UPDATE PartSupp
        SET Availqty = Availqty - L_valuediff
        WHERE Partkey = NEW.Partkey AND Suppkey = NEW.Suppkey;
    END;
ELSE
    /* 如果可用数量不能满足订单订购数量, 则插入过程异常中断。*/
    RAISE EXCEPTION 'Available Quantity is NOT ENOUGH';
END IF;
END;

```

③ 在 Lineitem 表上定义一个 BEFORE DELETE 触发器, 当删除订单明细时, 该订单明细项订购的数量要归还对应的零件供应记录。

```

CREATE OR REPLACE TRIGGER TRI_Lineitem_Quantity_UPDATE
BEFORE DELETE ON Lineitem
FOR EACH ROW
AS
DECLARE
    L_valuediff INTEGER;
    L_availqty INTEGER;
BEGIN
    /* 获得删除订单明细项的订购数量*/
    L_valuediff = -OLD.Quantity;
    /* 修改当前订单明细项对应零件供应记录中的可用数量 */
    UPDATE PartSupp
    SET Availqty = Availqty - L_valuediff
    WHERE Partkey = NEW.Partkey AND Suppkey = NEW.Suppkey;
END;

```

验证触发器 TRI_Lineitem_Quantity_UPDATE

```

    /* 查看 1854 号订单第 1 个明细项的零件和供应商编号、订购数量、可用数量*/

SELECT L.Partkey, L.Suppkey, L.Quantity, PS.Availqty
FROM Lineitem L, PartSupp PS
WHERE L.Partkey = PS.partkey AND L.Suppkey = PS.Suppkey AND
    L.Orderkey = 1854 AND L.Linenum = 1;
/* 激活触发器: 修改 1854 号订单第 1 个明细项的订购数量*/
UPDATE Lineitem SET Quantity = Quantity + 5
WHERE Orderkey = 1854 AND Linenum = 1;

```

续表

/* 再次查看 1854 号订单第 1 个明细项的相关信息, 以验证触发器是否起作用*/

```
SELECT L.Partkey, L.Suppkey, L.Quantity, PS.Availqty
FROM Lineitem L, PartSupp PS
WHERE L.Partkey = PS.Partkey AND L.Suppkey = PS.Suppkey AND
      L.Orderkey = 1854 AND L.Linenumber = 1;
```

(6) 删除触发器

删除触发器

```
TRI_Lineitem_Price_UPDATE
DROP TRIGGER TRI_Lineitem_Price_UPDATE ON Lineitem;
```

实验总结:

触发器既可以实现复杂的用户自定义完整性约束, 也可以实现自动化数据库操作。当在一个表上定义多个触发器时, 要搞清楚触发器的执行顺序, 否则可能会得到不想要的结果。

3. 思考题

- ① 请设计一个 AFTER 触发器, 当 Lineitem 表中的 Quantity 变化时, 自动计算 Lineitem 表中的 Extendedprice 值, 同时也要修改 PartSupp 中的 Availqty 值 (提示: $\text{Extendedprice} = \text{Quantity} * \text{Part.Retailprice}$)。
- ② 尝试给一个表设计多个触发器, 探索多个触发器的执行顺序。

第7章实验 数据库设计

数据库设计与应用开发实验是一个大型实验项目, 总共包括 6 个实验, 即实验 7.1 (见表 7)、实验 8.1~实验 8.5。其中, 实验 7.1 针对第 7 章“数据库设计”中的数据库概念结构设计、逻辑结构设计和物理结构设计等内容进行实验; 实验 8.1~实验 8.4 针对第 8 章“数据库编程”内容, 实验 8.5 则是一个综合性的数据库大作业。

表 7 “数据库设计”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应《概论》章节
实验 7.1	数据库设计实验	必修	适中	4	第 7 章

1. 实验内容和要求

掌握数据库设计基本方法及数据库设计工具。掌握数据库设计的基本步骤, 包括数据库概念结构设计、逻辑结构设计、物理结构设计、数据库模式 SQL 语句生成; 能够选择并使用数据